

Loan Approval Prediction — Course Notes

Everything you need to know, from zero, to do this assignment properly.

0. First, understand what you've actually been assigned

Read the two pages again. Notice something: the problem statement talks about building an ML model, but the *deliverables* are an **EDA Report** and a **Visualization Report**. There is no "train a classifier" requirement. No accuracy score. No train/test split.

This is deliberate. You are being assigned the part of the job that actually consumes 70–80% of a real data scientist's time and that most students skip past in a hurry to call `.fit()`. The ML framing is context — it tells you *why* you're cleaning this data. Everything you do should be answerable with: "because a model trained on this would otherwise learn something false."

So the real question you're answering across the whole notebook is:

What in this data determines whether a loan gets approved, and what condition is this data in before anyone tries to model it?

Keep that sentence at the top of your notebook. Every plot, every cleaning step, should serve it.

A second thing to notice: the phrase "write inference from each plot." Producing nine charts takes an afternoon. Producing nine charts with a paragraph each explaining what they mean is the assignment. Most submissions will be charts with no commentary and will be graded accordingly.

1. Environment setup

Install Anaconda. Download from anaconda.com, run the installer, accept defaults. This gives you Python, Jupyter, pandas, NumPy, matplotlib, seaborn, and scikit-learn in one shot. Do not install these individually with pip while you're learning — version conflicts will eat a day you don't have.

Launch Jupyter Notebook. Open Anaconda Prompt (Windows) or Terminal (Mac/Linux):

A browser tab opens showing your file system. Navigate to a folder, click *New* → *Python 3*. Put your CSV in the same folder as the notebook. This saves you from path headaches.

Notebook basics you need:

- A notebook is a list of **cells**. Cells are either **Code** or **Markdown** (switch with the dropdown, or press `Esc` then `Y` for code, `M` for markdown).
- `Shift + Enter` runs the current cell and moves down. `Ctrl + Enter` runs and stays.
- Cells share state. If you run cell 5, then edit cell 2 and run it, cell 5's variables are stale until you re-run it. This causes more student confusion than any other single thing.
- Before you submit: **Kernel → Restart & Run All**. If it doesn't run top-to-bottom cleanly, it's broken, and your examiner will find out in ten seconds.
- The last expression in a cell displays automatically. `df.head()` alone prints; you don't need `print()`.

Markdown cells are not optional here. Your assignment asks for a *report*. A report is prose with evidence embedded, not a wall of code. Structure:

```
## 3. Handling Missing Values
```

```
The `self_employed` column had 4.2% missing values. Since it is  
categorical and the missingness appears unrelated to loan status,  
I imputed with the mode rather than dropping rows...
```

Then the code cell. Then the output. Then your reading of the output.

2. Python, only the parts you need

If you're shaky on Python, this is the whole subset required:

```
# Variables and types
n = 4269                      # int
rate = 0.62                   # float
name = "cibil_score"          # str
approved = True               # bool

# Lists — ordered, mutable, square brackets
```

```

cols = ["income", "loan_amount", "cibil_score"]
cols[0]          # "income" - indexing starts at 0
cols[-1]         # "cibil_score" - negative counts from the end
cols[0:2]        # ["income", "loan_amount"] - end is exclusive

# Dictionaries - key/value pairs, curly braces
mapping = {"Approved": 1, "Rejected": 0}
mapping["Approved"]  # 1

# Loops
for col in cols:
    print(col)

# Functions
def iqr_bounds(series):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    return q1 - 1.5 * iqr, q3 + 1.5 * iqr

# Imports - always at the top of your notebook, one cell
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

```

Two idioms that appear constantly in pandas code and confuse beginners:

List comprehension — a loop that builds a list in one line:

```
numeric_cols = [c for c in df.columns if df[c].dtype != "object"]
```

Read it right-to-left: "for each c in df.columns, if it's not text, keep c."

Method chaining — each method returns something you can call the next method on:

```
df["education"].str.strip().value_counts()
```

Take the column → strip whitespace from each string → count how many of each value.

That's genuinely enough Python. Everything else is library API.

3. NumPy in sixty seconds

NumPy is the array library pandas is built on. You'll touch it rarely but directly:

```
np.nan          # the missing-value marker. Not zero. Not None.
np.log(x)       # natural log — useful for skewed money columns
np.where(cond, a, b)  # vectorized if/else
```

The one concept worth understanding: **vectorization**. In plain Python you'd loop over rows. In NumPy/pandas you operate on the whole column at once:

```
df["total_assets"] = df["residential_assets_value"] + df["commercial_assets_value"]
```

No loop. This is faster and it's how idiomatic pandas is written. If you find yourself writing `for i in range(len(df))`, stop — there's a vectorized way.

4. Pandas: the core of everything

4.1 The two objects

- **Series** — one column. A labelled 1-D array.
- **DataFrame** — a table. A dictionary of Series that share an index.

```
df = pd.read_csv("loan_approval_dataset.csv")
df["cibil_score"]          # a Series
df[["cibil_score", "income_annum"]]  # a DataFrame (note double brackets)
```

4.2 Loading and first inspection

```
df = pd.read_csv("loan_approval_dataset.csv")

df.head(5)          # first 5 rows          ← assignment requirement
df.tail(5)          # last 5 rows           ← assignment requirement
df.shape            # (rows, columns)       ← assignment requirement
df.info()           # dtype + non-null count per column ← requirement
df.describe()       # count/mean/std/min/quartiles/max for numeric cols
df.dtypes           # just the types
df.columns          # the column names
```

`df.info()` is the single most informative command in pandas. It answers "how many rows, what columns, what types, how many nulls" in one call. Run it after every major transformation to confirm you did what you intended.

4.3 The whitespace trap — read this before anything else

Public loan datasets of this kind are notorious for **leading spaces in column names and in string values**. Your column isn't `education`, it's `" education"`. Your value isn't `"Graduate"`, it's `" Graduate"`. Every filter you write silently returns zero rows and you spend two hours convinced pandas is broken.

Check first:

```
print(list(df.columns))      # look for spaces inside the quotes
```

Fix immediately, before anything else:

```
df.columns = df.columns.str.strip()

# and for every text column:
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip()
```

Do this in your very first cleaning cell and mention it in markdown. Noticing this is exactly the kind of thing that separates a careful submission from a sloppy one.

4.4 Renaming columns

The assignment explicitly asks for meaningful names. Use a dictionary:

```
df = df.rename(columns={
    "cibil_score": "credit_score",
    "income_annum": "annual_income",
    "loan_term": "loan_term_years",
    "no_of_dependents": "dependents",
})
```

Be careful not to rename into meaninglessness. `credit_score` is better than `cibil_score` for a general audience; `x1` is worse than both. Rename for clarity, and say in markdown why each change helps.

4.5 Selecting rows and columns

```
df.loc[0]                # row by label
df.iloc[0]               # row by position
df.loc[0:4, ["credit_score"]] # rows 0-4 (inclusive!), one column
df.iloc[0:5, 0:3]         # rows 0-4, cols 0-2 (exclusive)
```

```
# Boolean filtering – the workhorse
df[df["credit_score"] > 700]
df[(df["credit_score"] > 700) & (df["loan_status"] == "Approved")]
```

Note: `&` and `|`, `not` and `/or`. Each condition must be in its own parentheses. This trips up everyone once.

4.6 Grouping and counting

These two produce most of your insights:

```
df["loan_status"].value_counts()           # raw counts
df["loan_status"].value_counts(normalize=True) # proportions

df.groupby("education")["loan_status"].value_counts(normalize=True)
df.groupby("loan_status")["credit_score"].mean()
df.groupby("loan_status")[["annual_income", "loan_amount"]].agg(["mean", "median"])
```

`groupby` is: split the data into groups → compute something per group → stick the results back together. Almost every "does X affect approval?" question is a `groupby`.

4.7 Copy vs view — the warning you will see

```
subset = df[df["credit_score"] > 700]
subset["flag"] = 1           # SettingWithCopyWarning
```

Pandas isn't sure whether you meant to modify `df` or a copy. Be explicit:

```
subset = df[df["credit_score"] > 700].copy()
subset["flag"] = 1           # fine
```

Also make a deliberate copy before destructive experiments so you can always fall back:

```
df_raw = df.copy()           # pristine, never touched again
```

5. Types of data — and why the assignment leads with it

Your syllabus lists "Types of Data and how to work with them" first. It's listed first because the type of a variable dictates every downstream choice: which imputation, which encoding,

which plot, which summary statistic.

Numerical

- *Continuous* — can take any value in a range. Income, loan amount, asset values.
- *Discrete* — countable whole units. Number of dependents, loan term in years.

Categorical

- *Nominal* — categories with no inherent order. Self-employed yes/no. Loan status approved/rejected.
- *Ordinal* — categories with a meaningful order. Education (in some framings), a credit-rating band like Poor < Fair < Good < Excellent.

Why this matters concretely:

Question	Numeric	Nominal	Ordinal
Fill missing with?	mean or median	mode	mode or median category
Encode how?	already numeric	one-hot	integer, order preserved
Summarize with?	mean, median, std	frequency counts	counts, median
Plot with?	histogram, box	bar, pie	ordered bar

There is also a subtler distinction worth one line in your report: **interval vs ratio**. Ratio data has a true zero, so ratios are meaningful — ₹200,000 income is genuinely twice ₹100,000. Interval data has no true zero — temperature in Celsius. Almost everything in this dataset is ratio, which is why sums, ratios and log transforms are all legitimate here.

One derived feature worth creating, because it demonstrates you understand the domain:

```
df["loan_to_income"] = df["loan_amount"] / df["annual_income"]
```

A ₹5,000,000 loan means something completely different to someone earning ₹1,000,000 versus ₹10,000,000. Raw loan amount alone is a weak signal; the ratio is how lenders actually think. Similarly, total assets as a sum of the four asset columns.

6. Missing values

6.1 Finding them

```
df.isnull().sum()                                # count per column
df.isnull().sum()[df.isnull().sum() > 0]         # only columns that have any
df.isnull().mean() * 100                         # percentage per column
```

6.2 The important conceptual bit: *why* is it missing?

Statisticians distinguish three mechanisms, and your examiner will be pleased to see the vocabulary:

- **MCAR** (Missing Completely At Random) — missingness unrelated to anything. A sensor glitched. Dropping rows is safe, just wasteful.
- **MAR** (Missing At Random) — missingness related to *other observed* variables. Self-employed applicants are more likely to leave income blank. Imputation using the related variable is defensible.
- **MNAR** (Missing Not At Random) — missingness related to *the missing value itself*. People with terrible credit scores decline to report them. This is dangerous: any imputation biases the data, and the *fact of being missing* is itself informative. The right move is often to add a `was_missing` indicator column.

You can't prove which mechanism you have from data alone, but you can reason about it, and that reasoning is what earns marks.

6.3 The methods

```
# Delete
df.dropna()                                # drop any row with any null
df.dropna(subset=["income"])               # drop only if income is null
df.dropna(axis=1)                         # drop columns — rarely what you want

# Impute
df["income"] = df["income"].fillna(df["income"].mean())
df["income"] = df["income"].fillna(df["income"].median())
df["self_employed"] = df["self_employed"].fillna(df["self_employed"].mode()[0])
```

Mean vs median vs mode — the decision rule:

- **Mean** for roughly symmetric numeric data. It uses all information but is dragged around by outliers.
- **Median** for skewed numeric data. Income and loan amounts are almost always right-

skewed (a few very large values), so **median is usually correct for money columns**. Say this explicitly in your report.

- **Mode** for categorical data. There is no "average" of Graduate and Not Graduate.

`.mode()` returns a Series (there can be ties), hence the `[0]`.

Group-wise imputation is more sophisticated and cheap to do:

```
df["income"] = df.groupby("education")["income"].transform(
    lambda s: s.fillna(s.median())
)
```

Fill each person's missing income with the median income *of their education group*. One line, visibly better reasoning.

6.4 The instruction you must not miss

Your assignment says: *"Please insert some data by your own if existing data is not suffice to demonstrate above requirements."*

This is your examiner telling you, in advance, that **the dataset is probably already clean**. No nulls. No duplicates. If you just run `isnull().sum()`, see all zeros, and write "no missing values found," you have demonstrated none of the required skills and you will lose those marks.

What to do instead — inject damage deliberately, on a copy, and repair it:

```
df_dirty = df.copy()

# Inject nulls reproducibly
np.random.seed(42)
idx = np.random.choice(df_dirty.index, size=200, replace=False)
df_dirty.loc[idx[:100], "annual_income"] = np.nan
df_dirty.loc[idx[100:], "self_employed"] = np.nan

# Inject duplicates
df_dirty = pd.concat([df_dirty, df_dirty.sample(50, random_state=42)],
                    ignore_index=True)

print("Nulls introduced:\n", df_dirty.isnull().sum())
print("Duplicate rows:", df_dirty.duplicated().sum())
```

`np.random.seed(42)` makes it reproducible — same result every run, so your notebook's output matches its text. Then demonstrate all three repair strategies (drop / mean / median /

mode), compare what each does to the distribution, and state which you'd keep and why.

Frame this honestly in markdown: *"The source data contained no missing values. To demonstrate the required handling techniques, I introduced controlled missingness as follows..."* Honesty here reads as competence, not as cheating.

7. Duplicates

```
df.duplicated().sum()          # how many full-row duplicates
df[df.duplicated(keep=False)]  # show all copies, including first
df = df.drop_duplicates()
df = df.drop_duplicates(subset=["loan_id"]) # duplicate IDs specifically
```

Two different problems, worth distinguishing in your write-up:

- **Exact duplicate rows** — usually a data pipeline error, safe to drop.
- **Duplicate identifiers with different values** — the same `loan_id` appearing twice with different amounts. This is a *data integrity* problem, not a cleaning problem. You can't just drop one at random; you'd need to know which is authoritative. Flag it, don't silently resolve it.

Always report counts before and after. `print(f"Rows before: {len(df)}, after: {len(df_clean)}")` .

8. Encoding categorical data

Models consume numbers. Text columns must become numeric. There are two methods and choosing wrong is a genuine mistake, not a style preference.

8.1 Label / ordinal encoding

Maps each category to an integer.

```
df["loan_status_encoded"] = df["loan_status"].map({"Rejected": 0, "Approved": 1})

# or with sklearn
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["education_encoded"] = le.fit_transform(df["education"])
print(dict(zip(le.classes_, le.transform(le.classes_)))) # show the mapping
```

Use this for: binary columns (yes/no, approved/rejected — only two values, so no false ordering is possible) and genuinely ordinal columns where you control the order:

```
order = {"Poor": 0, "Fair": 1, "Good": 2, "Excellent": 3}
df["score_band_encoded"] = df["score_band"].map(order)
```

8.2 One-hot encoding

Creates one binary column per category.

```
df = pd.get_dummies(df, columns=["education"], drop_first=True)
```

Use this for: nominal columns with more than two categories.

8.3 The trap

If you label-encode a nominal column with three or more categories — say property area as Urban=0, Rural=1, Semiurban=2 — you have told the model that Semiurban > Rural > Urban, and that Semiurban is *twice* Rural. That ordering is fiction. A linear model will fit a slope through it and learn nonsense.

State this trade-off in your report. It is one of the clearest markers of someone who understands encoding rather than having memorised it.

`drop_first=True` removes one redundant column (if it's not Graduate, it's Not Graduate — the second column carries no new information). This avoids perfect multicollinearity, which breaks linear models. Worth one sentence.

9. Outliers

9.1 The IQR method

```
Q1 = df["annual_income"].quantile(0.25)
Q3 = df["annual_income"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["annual_income"] < lower) | (df["annual_income"] > upper)]
print(f"Outliers: {len(outliers)} ({len(outliers)/len(df)*100:.2f}%)")
```

Where does 1.5 come from? Tukey's convention. For normally distributed data it flags roughly the most extreme 0.7%. It is a heuristic, not a law — say so.

9.2 The z-score method

```
from scipy import stats
z = np.abs(stats.zscore(df["annual_income"]))
outliers_z = df[z > 3]
```

Z-score assumes approximate normality. Income is skewed, so IQR is the safer default here. Mention that you chose IQR *for that reason* — again, reasoning over recipe.

9.3 The judgment that actually matters

Here is where students lose marks by being too aggressive.

An outlier is **not** the same thing as an error. There are three cases, and they get three different treatments:

1. **Data error.** A negative asset value. An income of 0 with an approved loan. Loan term of 400 years. These are impossible. Investigate, then correct or remove.
2. **Legitimate extreme value.** A genuinely wealthy applicant earning fifty times the median. This is real and, for a bank, the most commercially interesting segment.
Deleting it destroys the information the model most needs.
3. **Ambiguous.** Unusual but not impossible. Flag it, keep it, note the sensitivity.

Financial data is right-skewed *by nature*. Wealth distributions have long tails. If you IQR-filter income aggressively you will delete every high-net-worth applicant and your "model" will only work for the middle of the market — a commercially useless result.

Check specifically for negative values in asset columns, which some versions of this dataset contain:

```
asset_cols = [c for c in df.columns if "asset" in c]
for c in asset_cols:
    n = (df[c] < 0).sum()
    if n:
        print(f"{c}: {n} negative values")
```

Negative assets are almost certainly a data-entry artefact. Finding this yourself and writing about it is worth more than any chart in the notebook.

Your options, in increasing order of aggression:

```
# 1. Keep and flag
df["income_is_outlier"] = (df["annual_income"] > upper).astype(int)

# 2. Cap / winsorize — pull extremes to the boundary, keep the row
df["income_capped"] = df["annual_income"].clip(lower, upper)

# 3. Transform — compress the tail instead of cutting it
df["income_log"] = np.log1p(df["annual_income"])

# 4. Remove — last resort, for genuine errors only
df = df[(df["annual_income"] >= lower) & (df["annual_income"] <= upper)]
```

`log1p` is $\log(1+x)$, which handles zeros safely. For skewed money columns it often turns an unreadable histogram into a clean, near-normal one — a genuinely impressive thing to show side by side.

Whatever you choose: **state the decision and the justification.** "I capped rather than removed, because extreme incomes are legitimate and represent 3.1% of applicants, of whom 71% were approved — a segment the bank cannot afford to discard."

10. Binning and normalization

10.1 Binning

Converting a continuous variable into categories.

```
# Equal-width bins with custom, domain-meaningful edges
df["score_band"] = pd.cut(
    df["credit_score"],
    bins=[300, 550, 650, 750, 900],
    labels=["Poor", "Fair", "Good", "Excellent"]
)

# Equal-frequency bins — each bin has the same number of rows
df["income_quartile"] = pd.qcut(
    df["annual_income"], q=4,
    labels=["Q1", "Q2", "Q3", "Q4"]
)
```

`cut` = equal *width*, you choose the edges. `qcut` = equal *count*, pandas chooses the edges.

Why bin at all? Interpretability. "Applicants scoring below 550 are approved 4% of the time" is a sentence a bank manager can act on. "There is a positive coefficient on credit score" is not. Binning also captures non-linear effects — if approval jumps sharply at a threshold rather than rising smoothly, bins reveal that and a linear model doesn't.

The cost is information loss: someone at 549 and someone at 551 are treated as different, and someone at 551 and 649 as identical. Note the trade-off.

For credit-score bins, use real-world CIBIL bands (roughly 300–549 poor, 550–649 fair, 650–749 good, 750–900 excellent) rather than arbitrary quartiles. Using domain knowledge instead of `qcut` defaults is exactly the kind of choice that gets noticed.

10.2 Normalization and standardization

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Min-Max → rescales to [0, 1]
mm = MinMaxScaler()
df[["income_scaled"]] = mm.fit_transform(df[["annual_income"]])

# Standard → mean 0, standard deviation 1 (a "z-score")
ss = StandardScaler()
df[["income_std"]] = ss.fit_transform(df[["annual_income"]])
```

- **Min-Max** preserves the shape of the distribution, bounds it to [0,1]. Sensitive to outliers — one extreme value squashes everything else into a narrow band.
- **Standardization** centres on the mean. Doesn't bound the range. More robust to outliers, and the default for most algorithms.

Why bother? Algorithms that measure distance (KNN, K-means, SVM) or use gradient descent (neural networks, logistic regression) are dominated by whichever feature has the largest raw numbers. Income in the millions will overwhelm loan term in years — not because it matters more, but because its units are bigger. Tree-based models (decision trees, random forests) are immune, since they split on thresholds rather than distances. Say which camp your intended model is in.

10.3 The sequencing rule — do not get this wrong

Do your visualizations on cleaned but unscaled data. Put scaling in its own section at the end.

If you scale first, your histogram x-axis reads 0.0 to 1.0 and no human can tell whether that's ₹200,000 or ₹20,000,000. Your box plots become uninterpretable. Your story evaporates.

Scaling is model preparation, not analysis.

The correct order for the whole notebook:

```
load → strip whitespace → rename → inspect →  
handle nulls → drop duplicates → fix data errors →  
create derived features → EXPLORE AND VISUALIZE →  
encode → treat outliers → bin → scale → export clean CSV
```

11. Visualization

11.1 The two libraries

matplotlib is the engine — verbose, total control. **seaborn** sits on top — short commands, better defaults, statistical plots built in. Use seaborn for the plot, matplotlib for the labels and layout.

Set this up once at the top:

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
sns.set_theme(style="whitegrid")  
plt.rcParams["figure.figsize"] = (10, 6)  
plt.rcParams["figure.dpi"] = 100
```

The universal pattern:

```
plt.figure(figsize=(10, 6))  
sns.histplot(data=df, x="credit_score", hue="loan_status", bins=30)  
plt.title("Credit Score Distribution by Loan Status")  
plt.xlabel("Credit Score")  
plt.ylabel("Number of Applicants")  
plt.tight_layout()  
plt.show()
```

Every single plot gets a title and axis labels. An unlabelled chart is not evidence, it's decoration. `tight_layout()` stops labels from being clipped.

11.2 The nine required plots — what each is *for*

Don't think "I need to produce a pie chart." Think "what question does a pie chart answer,

and do I have that question?" Then answer it about *this* data.

Basic / line plots — show a trend across an ordered dimension.

```
approval_by_score = df.groupby("score_band")["loan_status_encoded"].mean()
approval_by_score.plot(kind="line", marker="o")
```

Ask: how does approval rate move across score bands? A line plot is only honest when the x-axis has a natural order — don't line-plot across unordered categories.

Bar plots — compare a quantity across categories.

```
sns.barplot(data=df, x="education", y="loan_status_encoded", errorbar=None)
sns.countplot(data=df, x="loan_status", hue="self_employed")
```

Ask: does approval rate differ by education, self-employment, dependents? `barplot` shows a mean by default; `countplot` shows frequency. Know which you're showing.

Histograms — the distribution of one continuous variable.

```
sns.histplot(data=df, x="annual_income", bins=40, kde=True)
```

Ask: is income skewed? Bimodal? Are there suspicious spikes at round numbers? Adjust `bins` — too few hides structure, too many is noise. `kde=True` overlays a smoothed density curve.

Box plots — distribution summary, and the sharpest outlier-detection visual you have.

```
sns.boxplot(data=df, x="loan_status", y="credit_score")
```

Anatomy: box spans Q1 to Q3, line inside is the median, whiskers reach 1.5×IQR, dots beyond are flagged outliers. *Ask:* do approved and rejected applicants have visibly different score distributions? If the boxes barely overlap, that feature is powerfully predictive — and you can say so from the picture alone. Pair each box plot with a violin plot if you want to show the shape, not just the summary.

Area plots — cumulative or composed magnitude over an ordered axis.

```
pd.crosstab(df["score_band"], df["loan_status"]).plot(kind="area", stacked=True)
```

Ask: how does the composition of approved vs rejected shift as scores rise? Area plots suit part-to-whole change; they're a weak choice for anything else, and it's fine to say in your report that this was the least informative chart for this dataset. Noting that a standard tool

didn't suit the data is a legitimate finding.

Scatter plots — relationship between two continuous variables.

```
sns.scatterplot(data=df, x="annual_income", y="loan_amount",  
                hue="loan_status", alpha=0.5)
```

Ask: do income and requested loan amount move together? Do rejections cluster in a particular region? `alpha` makes overlapping points semi-transparent so you can see density. With 4,000+ points you need it.

Hexagonal bin plots — a scatter plot when there are too many points to see anything.

```
df.plot(kind="hexbin", x="annual_income", y="loan_amount",  
      gridsize=25, cmap="Blues")
```

Ask: where is the data actually concentrated? Scatter shows you *whether* points overlap; hexbin shows you *how much*. Use it on your densest pair of variables and explicitly compare it to the equivalent scatter — that comparison is itself an insight about visualization choice.

Pie plots — part of a whole, and only when you have few categories.

```
df["loan_status"].value_counts().plot(kind="pie", autopct="%1.1f%%")  
plt.ylabel("")
```

Ask: what's the class balance? This matters enormously downstream — if 62% are approved, a model that blindly predicts "approved" for everyone scores 62% accuracy while being worthless. That observation is one of the most valuable sentences you can write in this report.

Professional caveat worth including: pie charts are poor at precise comparison, because humans judge angles badly. Use them for two or three categories at most. Saying this shows judgment.

Heatmaps — correlation matrix, or any 2-D grid of values.

```
plt.figure(figsize=(12, 9))  
corr = df.select_dtypes(include=np.number).corr()  
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)  
plt.title("Correlation Matrix of Numeric Features")
```

Ask: which features correlate with loan status? Which correlate with *each other*? The second question matters as much as the first — if two asset columns correlate at 0.95, they

carry the same information and you'd keep one.

Read the numbers properly: correlation runs -1 to $+1$, measures **linear** association only, and **correlation is not causation**. A near-zero correlation does not mean "no relationship" — it means no *linear* relationship. A perfect U-shape has correlation zero.

Also try `pd.crosstab()` inside a heatmap for two categoricals:

```
sns.heatmap(pd.crosstab(df["score_band"], df["loan_status"]),
             annot=True, fmt="d", cmap="YlOrRd")
```

11.3 How to write an inference (the graded part)

Bad: *"This chart shows the credit score distribution."* That describes the chart. Anyone can see the chart.

Good, in four moves:

1. **What you see** — "Approved applicants have a median credit score of 711 versus 429 for rejected, with almost no overlap between the interquartile ranges."
2. **What it means** — "Credit score is by far the dominant factor in this dataset's approval decisions; approval appears to be driven by a hard threshold near 550 rather than a gradual effect."
3. **Why it matters / caveat** — "This suggests the underlying decisions were largely rule-based rather than holistic. A model trained here will effectively learn that rule, and will not generalise to a lender using different criteria."
4. **What next** — "Worth checking whether income or asset value has any effect *within* the high-score group, or whether score alone explains everything."

Two to four sentences per plot. Nine plots. That's your Visualization Report.

12. Storytelling with data

Your last syllabus bullet. The difference between a report and a pile of charts is a **through-line**.

Structure your notebook as an argument, not a checklist:

1. **Setup** — what the bank wants to know, what the data contains, how many rows and columns.

2. **Data quality** — what was wrong and what you did about it. Be specific and quantified.
3. **The single biggest finding** — lead with it, don't bury it in chart seven.
4. **Supporting findings** — features that matter less, features that surprisingly don't matter at all.
5. **Non-findings** — "I expected income to predict approval; it barely does once credit score is accounted for." Negative results are real results and most students omit them.
6. **Recommendation** — what should the bank actually do, and what should whoever builds the model know before they start.

Three principles:

- **Lead with the conclusion.** Executives read the first paragraph. Analysts read the rest.
- **Quantify everything.** "Higher scores help" is weak. "Applicants above 600 are approved 94% of the time versus 6% below 550" is a finding.
- **State your uncertainty.** "This dataset appears synthetic or heavily rule-generated; findings may not transfer to real applicant populations" is a mature observation, not an admission of failure. If your correlation heatmap shows one variable dominating everything and the others near zero, that's a strong hint, and noticing it is exactly what a professional would flag.

13. The seven-day plan

Day 1 — Environment and orientation. Install Anaconda, open Jupyter, load the CSV. Run `head, tail, shape, info, describe, dtypes`. Check and strip whitespace in column names and string values. Rename columns. Write your first markdown cells describing what the dataset contains. *End of day: you should be able to say what every column means in plain English.*

Day 2 — Missing values and duplicates. Run the null and duplicate checks. If clean (likely), inject controlled damage with a fixed seed. Demonstrate drop / mean / median / mode. Compare distributions before and after imputation. Document the mechanism reasoning (MCAR/MAR/MNAR). *End of day: EDA report bullets 1–7 done.*

Day 3 — Encoding, outliers, derived features. Encode binaries by mapping, nominals by one-hot, and write the paragraph about why. Run IQR on every money column. Check for negative asset values. Decide cap vs remove vs flag per column and justify each. Create `loan_to_income` and `total_assets`. *End of day: EDA report complete except binning/scaling.*

Day 4 — Plots, first half. Basic/line, bar, histogram, box, area. Write the inference immediately after each plot, while you still remember what you noticed. Do not batch the writing to the end — you will forget, and it will show.

Day 5 — Plots, second half. Scatter, hexbin, pie, heatmap. Same discipline. Then binning (`cut` with CIBIL bands) and scaling (Min-Max and Standard, in their own final section).

Day 6 — Assemble the story. Reorder cells into a logical narrative. Add an executive summary at the top and a conclusion at the bottom. Tidy every title and axis label. Export the cleaned dataset (`df.to_csv("loan_approval_cleaned.csv", index=False)`). Kernel → Restart & Run All, top to bottom, no errors.

Day 7 — Buffer. Something will break. A package won't install, a chart won't render, the notebook won't restart cleanly. Leave the day empty. If nothing breaks, use it to improve your weakest inferences.

14. How students lose marks on this exact assignment

- Charts with no inference text.
- Missing titles or axis labels.
- Writing "no nulls found" and moving on, ignoring the explicit instruction to insert data.
- Scaling before plotting, making every chart unreadable.
- Deleting all high-income applicants as "outliers" without justification.
- Label-encoding a multi-category nominal column and inventing a false ordering.
- A notebook that doesn't run top-to-bottom after Restart & Run All.
- No markdown — code and output only, no report.
- Never mentioning class imbalance, which is the single most consequential fact about a binary target.
- Copying a Kaggle notebook. Your examiner has read the popular ones. The tell is always the inference text — copied notebooks have generic commentary that doesn't match the specific numbers on screen.

15. Deliverables checklist

- One `.ipynb` notebook, runs cleanly top to bottom
 - Markdown headings and prose throughout, not just code
 - `head(5)` and `tail(5)` shown
 - Row/column count, per-column name, dtype and size shown
 - Columns renamed, reason given
 - Null columns identified; drop / mean / median / mode all demonstrated
 - Duplicates identified and removed, counts before and after
 - Categorical encoding done, with label vs one-hot justified
 - Outliers detected (IQR), treatment decided and justified per column
 - Binning done with meaningful domain bands
 - Normalization done, in its own section, after the visuals
 - All nine plot types present, each titled, labelled, and interpreted
 - Executive summary at the top, conclusion at the bottom
 - Cleaned dataset exported to CSV
-

16. If you get stuck

Read the error message from the **bottom up**. The last line names the error type and is usually the whole answer.

- `KeyError: 'income'` — that column name doesn't exist. Run `list(df.columns)`. It's almost always whitespace or a typo.
- `SettingWithCopyWarning` — add `.copy()` where you made the subset.
- `ValueError: could not convert string to float` — you're doing arithmetic on a text column. Check `df.dtypes`.
- Plot doesn't appear — add `plt.show()`.
- Weird stale results — Kernel → Restart & Run All.

And when something looks surprising in the data, don't reach for a fix straight away. Sit with it for a minute and ask what would have to be true of the world for that to be the real answer. That habit is the whole job.

